

Student Name	Sarvesh Naik
SRN No	31241943
Roll No	33
Program	Computer Engineering
Year	Third Year
Division	F
Subject	Artificial Intelligence Lab
Assignment No	3

Assignment 3

Problem Statement: Implement the Hill Climbing technique to solve the 8 puzzle problem

Objective:

The objective of this experiment is to implement the Hill Climbing technique to solve the 8-puzzle problem. The algorithm uses a heuristic function (such as misplaced tiles or Manhattan distance) to evaluate neighboring states and moves step by step toward a better state. The aim is to reach the goal configuration by continuously selecting the best neighboring state until the solution is found or no better move is possible.

What are Hurestics and Hurestic Functions:

Heuristics are problem-solving strategies or rules of thumb used to make decisions faster by reducing the search space. In search problems, heuristics help guide the algorithm toward the goal without exploring all possible states. They provide an estimate of how close a current state is to the goal.

A **Heuristic Function**, usually represented as $h(n)$, is a mathematical function that estimates the cost or distance from a current state (n) to the goal state. It helps the algorithm choose the most promising path. A good heuristic function improves efficiency by reducing unnecessary exploration and speeding up the search process.

Theory & Working:

Hill Climbing is a heuristic search algorithm used to solve optimization and search problems. It is a local search technique that continuously moves toward a better state based on a heuristic value. The main idea is to start from an initial state and repeatedly move to the neighboring state that has a better evaluation value until the goal is reached or no better neighbor exists.

Hill Climbing uses a heuristic function $h(n)$ to measure how close the current state is to the goal. The heuristic function provides an estimate of the cost or distance from the current state to the goal state. The algorithm always selects the neighbor with the best (lowest or highest, depending on the problem) heuristic value.

However, Hill Climbing may face problems such as:

- Local maxima – Reaches a better state but not the global best.
- Plateau – All neighboring states have the same heuristic value.
- Ridge problem – Requires moving in multiple directions to improve.

Working of Hill Climbing:

1. Start with an initial state.
2. Calculate the heuristic value $h(n)$ of the current state.
3. Generate all possible neighboring states.
4. Evaluate each neighbor using the heuristic function.
5. Select the neighbor with the best heuristic value.
6. If the selected neighbor is better than the current state, move to it.
7. Repeat the process until:
 - The goal state is reached, or
 - No better neighboring state is available (algorithm stops).

Manhattan Distance (Explanation):

Manhattan Distance is a heuristic function used mainly in grid-based problems like the 8-puzzle. It calculates the total distance each tile is away from its correct goal position.

It is called "Manhattan" because it measures distance like moving in city blocks (only horizontal and vertical moves, no diagonal moves), similar to streets in Manhattan.

$$\Rightarrow \text{Manhattan Distance} = |x_1 - x_2| + |y_1 - y_2|$$

Pseudo Code:

HILL_CLIMBING(initial_state)

1. current $\leftarrow$ initial_state

2. LOOP

a. If current = GOAL

 return current

b. neighbors $\leftarrow$ Generate all possible moves of current

c. best $\leftarrow$ current

 best_h $\leftarrow$ heuristic(current)

d. For each neighbor in neighbors

 h $\leftarrow$ heuristic(neighbor)

 If h < best_h

 best $\leftarrow$ neighbor

 best_h $\leftarrow$ h

e. If best_h $\geq$ heuristic(current)

 return current // Local optimum reached

f. current $\leftarrow$ best

3. END LOOP

Code:

```
"""
Hill Climbing - 8-Puzzle (Terminal Heuristic Table)

=====

Run: python hill_climbing_table.py

"""

import copy

# --- GOAL & SETUP -----

GOAL = [[1,2,3],
        [8,0,4],
        [7,6,5]]

GOAL_POS = {val: (r, c)
             for r, row in enumerate(GOAL)
             for c, val in enumerate(row)}

MOVE_LABELS = {
    ( 1, 0): "Down  ↓",
    (-1, 0): "Up   ↑",
    ( 0, 1): "Right →",
    ( 0,-1): "Left  ←",
}
```

```
# — HELPERS —————

def heuristic(state):

    """Manhattan distance of every tile from its goal position."""

    d = 0

    for i in range(3):

        for j in range(3):

            v = state[i][j]

            if v != 0:

                gr, gc = GOAL_POS[v]

                d += abs(gr - i) + abs(gc - j)

    return d


def find_blank(state):

    for i in range(3):

        for j in range(3):

            if state[i][j] == 0:

                return i, j


def get_neighbors(state):

    x, y = find_blank(state)

    result = []

    for (dx, dy), label in MOVE_LABELS.items():

        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
```

```

        ns = copy.deepcopy(state)

        ns[x][y], ns[nx][ny] = ns[nx][ny], ns[x][y]

        result.append((ns, label))

    return result

def state_str_inline(state):

    """Single-line compact representation: [2,8,3|1,6,4|7,_,5]"""

    rows = []

    for row in state:

        rows.append(",".join(str(v) if v != 0 else "_" for v in row))

    return "[" + " | ".join(rows) + "]"

def state_str_grid(state):

    """3-line grid for printing."""

    lines = []

    for row in state:

        lines.append("  " + " ".join(str(v) if v != 0 else "_" for v in row))

    return "\n".join(lines)

# — HILL CLIMBING —————

def hill_climbing(initial):

    current    = copy.deepcopy(initial)

    path       = [current]

    all_steps = []

```

```
for _ in range(100):

    current_h = heuristic(current)

    neighbours = get_neighbors(current)

    best_h, best_nb = current_h, None

    nb_info = []

    for nb, lbl in neighbours:

        h = heuristic(nb)

        nb_info.append({"state": nb, "label": lbl, "h": h, "chosen": False})

        if h < best_h:

            best_h, best_nb = h, nb

    # mark chosen

    if best_nb is not None:

        for ni in nb_info:

            if ni["state"] == best_nb:

                ni["chosen"] = True

                break

    all_steps.append({

        "current": copy.deepcopy(current),

        "current_h": current_h,

        "neighbours": nb_info,

    })
```

```
    if current == GOAL:

        return path, all_steps, "goal"

    if best_nb is None or best_h >= current_h:

        return path, all_steps, "local_optimum"

    current = best_nb

    path.append(copy.deepcopy(current))

    return path, all_steps, "max_steps"
```

```
# ——— TERMINAL TABLE PRINTER ———
```

```
# ANSI colour codes
```

```
RESET = "\033[0m"
```

```
BOLD = "\033[1m"
```

```
DIM = "\033[2m"
```

```
FG_WHITE = "\033[97m"
```

```
FG_BLACK = "\033[30m"
```

```
FG_GOLD = "\033[93m" # bright yellow
```

```
FG_ORANGE = "\033[38;5;208m"
```

```
FG_BLUE = "\033[94m"
```

```
FG_GREEN = "\033[92m"
```

```
FG_RED = "\033[91m"
```

```
FG_GRAY      = "\033[90m"

FG_CYAN      = "\033[96m"


BG_HEADER    = "\033[44m"      # blue bg

BG_CURRENT   = "\033[48;5;130m" # dark orange bg

BG_CHOSEN     = "\033[48;5;58m"  # dark gold/olive bg

BG_REJECT     = "\033[48;5;17m"  # dark navy

BG_SEP       = "\033[48;5;235m"  # very dark gray


def cell(text, width, align="left"):

    """Pad text to width."""

    if align == "right":

        return text.rjust(width)

    elif align == "center":

        return text.center(width)

    return text.ljust(width)


def print_heuristic_table(all_steps, outcome):

    # Column widths

    W_STEP      = 6

    W_ROLE      = 9

    W_MOVE      = 10

    W_STATE     = 27

    W_H         = 5

    W_TILE      = 7    # per tile column (8 tiles shown)
```

```

TILES    = [1,2,3,4,5,6,7,8]

# total width

total = 2 + W_STEP + 3 + W_ROLE + 3 + W_MOVE + 3 + W_STATE + 3 + W_H + 3 +
W_TILE*len(TILES) + 2

bar      = "-" * total

dbar     = "=" * total

def hdr_row():

    parts = [

        " " + cell("Step",    W_STEP,    "center"),

        cell("Role",         W_ROLE,     "center"),

        cell("Move",         W_MOVE,     "center"),

        cell("State  [r0 | r1 | r2]", W_STATE, "center"),

        cell("h(n)", W_H, "center"),

    ]

    for t in TILES:

        parts.append(cell(f"T{t}dist", W_TILE, "center"))

    return " ".join(parts)

def sep_row(ch="-"):

    return ch * total

def data_row(step_lbl, role, move, state, h_val, bg, fg):

```

```

# per-tile distances

tile_dists = []

for t in TILES:

    found = False

    for i in range(3):

        for j in range(3):

            if state[i][j] == t:

                gr, gc = GOAL_POS[t]

                d = abs(gr-i) + abs(gc-j)

                tile_dists.append(str(d))

                found = True

    if not found:

        tile_dists.append("-")

parts = [

    " " + cell(step_lbl, W_STEP, "center"),

    cell(role, W_ROLE, "center"),

    cell(move, W_MOVE, "center"),

    cell(state_str_inline(state), W_STATE, "left"),

    cell(str(h_val), W_H, "center"),

]

for d in tile_dists:

    parts.append(cell(d, W_TILE, "center"))

line = " ".join(parts)

```

```

        return bg + fg + BOLD * (bg != BG_REJECT) + line + RESET

# — PRINT —————

print()

print(FG_WHITE + BOLD + "  " + dbar + RESET)

print(FG_CYAN + BOLD +

      "  HILL CLIMBING - 8-PUZZLE |  HEURISTIC TABLE |  Manhattan Distance
h(n)".center(total+2)

      + RESET)

print(FG_WHITE + BOLD + "  " + dbar + RESET)

print()

# column headers

print(BG_HEADER + FG_WHITE + BOLD + "  " + hdr_row() + "  " + RESET)

print(FG_GRAY + "  " + sep_row() + RESET)

for si, step in enumerate(all_steps):

    cur = step["current"]

    cur_h = step["current_h"]

# — CURRENT ROW —

print(data_row(f"S{si}", "CURRENT", "-", cur, cur_h,

              BG_CURRENT, FG_GOLD))

# — NEIGHBOUR ROWS —

```

```

        for ni in step["neighbours"]:

            role = ">> CHOSEN" if ni["chosen"] else "rejected"

            bg = BG_CHOSEN if ni["chosen"] else BG_REJECT

            fg = FG_GOLD if ni["chosen"] else FG_BLUE

            print(data_row("", role, ni["label"], ni["state"], ni["h"],

                            bg, fg))

# — STEP SEPARATOR —

print(FG_GRAY + " " + sep_row() + RESET)

# — SUMMARY —————

print()

outcome_txt = {

    "goal": FG_GREEN + BOLD + " ✓ GOAL REACHED —  $h(n) = 0$ " + RESET,

    "local_optimum": FG_RED + BOLD + " ✖ LOCAL OPTIMUM — no neighbour improves  $h(n)$ " + RESET,

    "max_steps": FG_RED + BOLD + " ⚠ MAX STEPS REACHED" + RESET,

}.get(outcome, outcome)

print(outcome_txt)

# colour key

print()

print(FG_GRAY + " Colour Key:" + RESET)

print(BG_CURRENT + FG_GOLD + BOLD + " CURRENT " + RESET +

      FG_GRAY + " Active node at that step" + RESET)

```

```

print(BG_CHOSEN + FG_GOLD + BOLD + "   CHOSEN   " + RESET +

      FG_GRAY + "   Best neighbour - algorithm moves here" + RESET)

print(BG_REJECT + FG_BLUE + "   rejected   " + RESET +

      FG_GRAY + "   Neighbour with h >= current h (not chosen)" + RESET)

print()

print(FG_GRAY + "   T1dist...T8dist = per-tile Manhattan distance contribution to
h(n)" + RESET)

print()

# — MAIN —————

if __name__ == "__main__":

    initial = [[2, 8, 3],

               [1, 6, 4],

               [7, 0, 5]]

    print(FG_CYAN + BOLD + "\n   Initial State:" + RESET)

    print(state_str_grid(initial))

    print(FG_CYAN + BOLD + "\n   Goal State:" + RESET)

    print(state_str_grid(GOAL))

    print(FG_GRAY + f"\n   Initial h(n) = {heuristic(initial)}\n" + RESET)

    path, all_steps, outcome = hill_climbing(initial)

    print_heuristic_table(all_steps, outcome)

```

```

print(FG_WHITE + BOLD + "  Solution Path  (" + str(len(path)-1) + " moves):" +
RESET)

for i, state in enumerate(path):

    h = heuristic(state)

    label = "START" if i == 0 else ("GOAL" if state == GOAL else f"Step {i}")

    print(FG_GOLD + f"\n  [{label}]  h = {h}" + RESET)

    print(state_str_grid(state))

```

Output:



